

Databases

Document Databases: Modelleren

Inhoud

- Wat is een document database
 - Verschillen met een relationele database
 - Voordelen & nadelen
 - Collecties & documenten
- Modelleren in document databases
 - Embedding vs. Referencing
- MongoDB
 - JSON Schema definiëren voor MongoDB

Document databases

Wat zijn document databases?

NoSQL



NoSQL is een verzamelnaam voor verschillende databases die **geen** gebruik maken van tabellen, kolommen en joins.

- Non-SQL / not only SQL
- Flexibelere structuur dan een SQL database
- Sneller voor grote hoeveelheden data
- Gedistribueerd → horizontaal schaalbaar
- Doorgaans **geen** (volledige) ACID ondersteuning
 - Atomic, Consistent, Isolation, Durability

NoSQL

NoSQL is een verzamelnaam voor verschillende databases die **geen** gebruik maken van tabellen, kolommen en joins.

- Non-SQL / no
- Flexibelere s
- Sneller voor
- Gedistribueerd → horizontaal schaalbaar
- Doorgaans **geen** (volledige) ACID ondersteuning
 - Atomic, Consistent, Isolation, Durability

Recent is de term “NewSQL” ingeburgerd geraakt, deze term omschrijft relationele databases die de schaalbaarheid van NoSQL databases aanbieden maar ook volledige ACID support aanbieden. [CockroachDB](#) is hier een voorbeeld van.

Document databases

Document databases zijn een subset van NoSQL databases die data bewaren die als *één* bestand (tekstbestand) weergegeven kunnen worden.

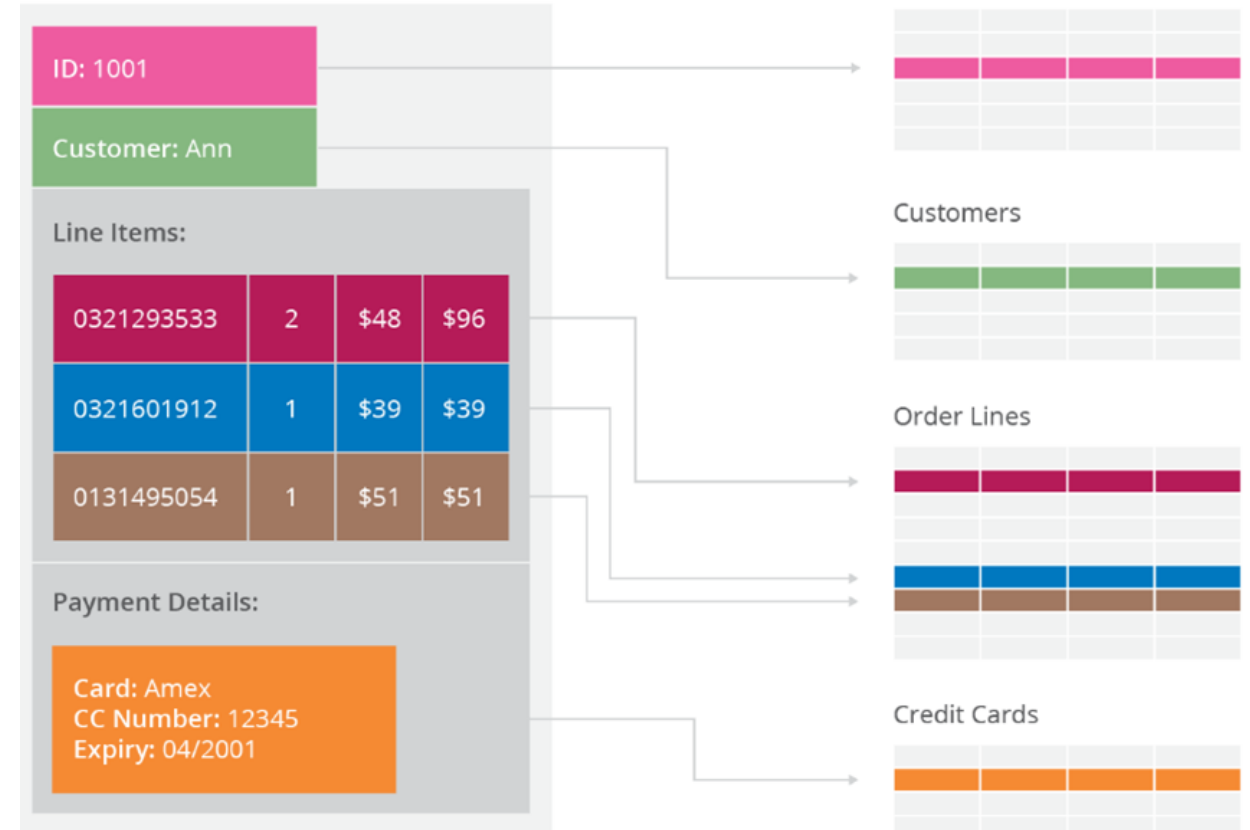
Kenmerken:

- Semigestructureerd data
 - Flexibel
 - Afwijkende properties
 - Geen schema/model nodig
- Dingen die samenhangen vormen één document
- Gestandaardiseerd formaat (JSON, BSON, YAML, XML, ...)
- Leunen aan bij objecten (OOP)
- Geen nood aan complexe ORM's

Relationeel vs. document

Relationele database

- 4 tabellen en 3 joins
- Query geeft 3 rijen terug
 - Duplicatie → ORM → Eén object



Relationeel vs. document

Order

id	...
1001	...

Customer

id	name
2560	Ann

Card

id	card	ccNumber	expiry
10256	Amex	12345	2001-04-01

Orderline

id	itemId	amount	price
56	0321293533	2	48
60	0321601912	1	39
61	0131495054	1	51

Relationeel vs. document

Join → duplicaten verwijderen (via ORM) → converteren naar een object → gebruiken in de view

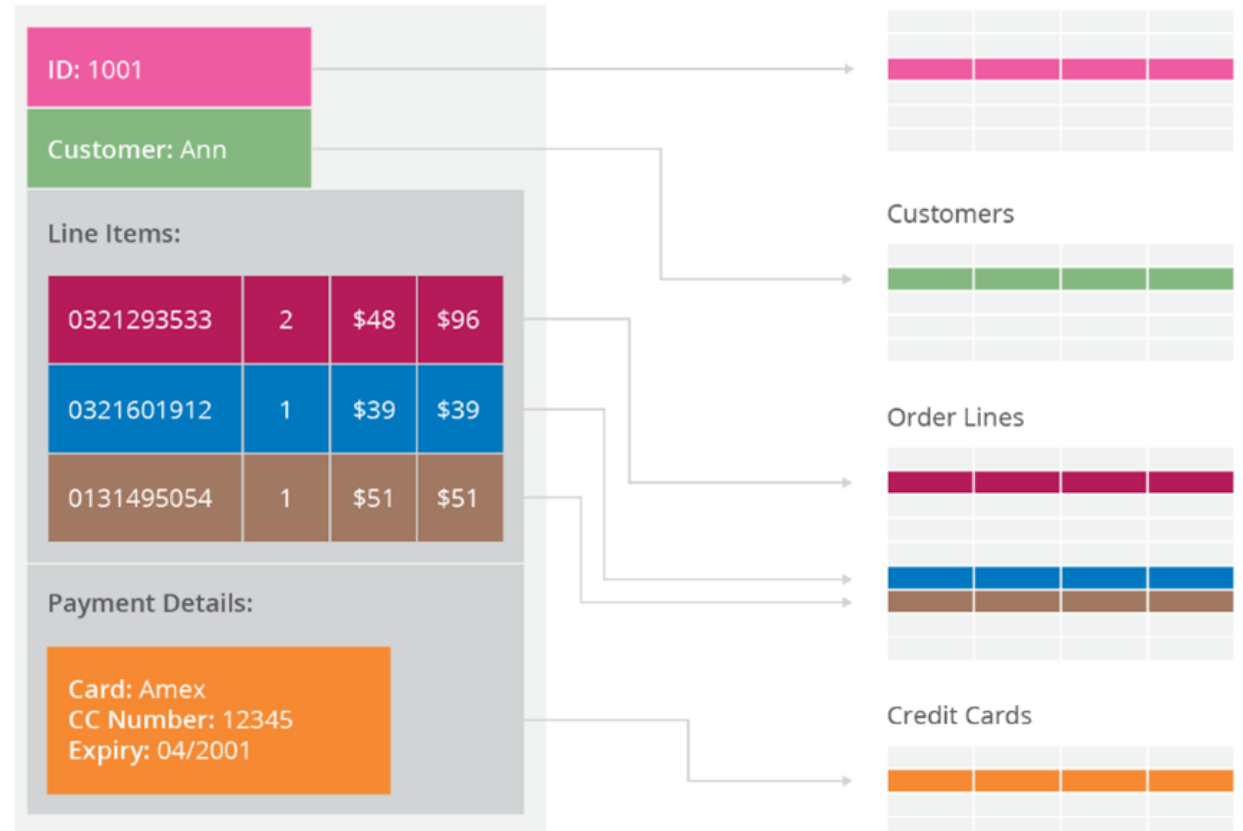
Na een join

Order.id	Customer.id	name	Orderline.id	itemId	amount	price	Card.id	card	ccNumber	expiry
1001	2560	Ann	56	0321293533	2	48	10256	Amex	12345	2001-04-01
1001	2560	Ann	60	0321601912	1	39	10256	Amex	12345	2001-04-01
1001	2560	Ann	61	0131495054	1	51	10256	Amex	12345	2001-04-01

Relationeel vs. document

Document database

- **Eén** document
- **Geen** joins
- Data wordt die samen opgevraagd wordt, wordt samen bewaard
 - Geen normalisatie



Relationeel vs. document

XML, JSON, YAML, ... → snelle
conversie naar een object (via ODM) →
gebruiken in view

Meestal **JSON**

- Zeer frequent gebruikt in JavaScript
en (REST) API's

```
{
  "id": 1001,
  "customer": "Ann",
  "lineItems": [
    {
      "item_id": 321293533,
      "amount": 2,
      "price": 48
    },
    {
      "item_id": 321601912,
      "amount": 1,
      "price": 39
    },
    {
      "item_id": 131495054,
      "count": 1,
      "price": 51
    }
  ],
  "paymentDetails": {
    "card": "Amex",
    "ccNumber": 12345,
    "expiry": "2001-04-01"
  }
}
```

Relationeel vs. document

XML, JSON, YAML, ... → snelle
conversie naar een object → gebruiken
in view

Meestal **JSON**

- {} duid een object/document aan

```
{
  "id": 1001,
  "customer": "Ann",
  "lineItems": [
    {
      "item_id": 321293533,
      "amount": 2,
      "price": 48
    },
    {
      "item_id": 321601912,
      "amount": 1,
      "price": 39
    },
    {
      "item_id": 131495054,
      "count": 1,
      "price": 51
    }
  ],
  "paymentDetails": {
    "card": "Amex",
    "ccNumber": 12345,
    "expiry": "2001-04-01"
  }
}
```

Relationeel vs. document

XML, JSON, YAML, ... → snelle
conversie naar een object → gebruiken
in view

Meestal **JSON**

- {} duid een object/document aan
- "key": "value" voor elk **string** attribuut

```
{
  "id": 1001,
  "customer": "Ann",
  "lineItems": [
    {
      "item_id": 321293533,
      "amount": 2,
      "price": 48
    },
    {
      "item_id": 321601912,
      "amount": 1,
      "price": 39
    },
    {
      "item_id": 131495054,
      "count": 1,
      "price": 51
    }
  ],
  "paymentDetails": {
    "card": "Amex",
    "ccNumber": 12345,
    "expiry": "2001-04-01"
  }
}
```

Relationeel vs. document

XML, JSON, YAML, ... → snelle
conversie naar een object → gebruiken
in view

Meestal **JSON**

- {} duid een object/document aan
- "key": "value" voor elk **string** attribuut
- "key": value voor elk ander datatype

```
{
  "id": 1001,
  "customer": "Ann",
  "lineItems": [
    {
      "item_id": 321293533,
      "amount": 2,
      "price": 48
    },
    {
      "item_id": 321601912,
      "amount": 1,
      "price": 39
    },
    {
      "item_id": 131495054,
      "count": 1,
      "price": 51
    }
  ],
  "paymentDetails": {
    "card": "Amex",
    "ccNumber": 12345,
    "expiry": "2001-04-01"
  }
}
```

Relationeel vs. document

XML, JSON, YAML, ... → snelle
conversie naar een object → gebruiken
in view

Meestal **JSON**

- {} duid een object/document aan
- "key": "value" voor elk **string** attribuut
- "key": value voor elk ander datatype
- [] voor arrays

```
{
  "id": 1001,
  "customer": "Ann",
  "lineItems": [
    {
      "item_id": 321293533,
      "amount": 2,
      "price": 48
    },
    {
      "item_id": 321601912,
      "amount": 1,
      "price": 39
    },
    {
      "item_id": 131495054,
      "count": 1,
      "price": 51
    }
  ],
  "paymentDetails": {
    "card": "Amex",
    "ccNumber": 12345,
    "expiry": "2001-04-01"
  }
}
```

Relationeel vs. document

XML, JSON, YAML, ... → snelle
conversie naar een object → gebruiken
in view

Meestal **JSON**

- {} duid een object/document aan
- "key": "value" voor elk **string** attribuut
- "key": value voor elk ander datatype
- [] voor arrays
- Attributen van elkaar scheiden met een komma (behalve het laatste)

```
{
  "id": 1001,
  "customer": "Ann",
  "lineItems": [
    {
      "item_id": 321293533,
      "amount": 2,
      "price": 48
    },
    {
      "item_id": 321601912,
      "amount": 1,
      "price": 39
    },
    {
      "item_id": 131495054,
      "count": 1,
      "price": 51
    }
  ],
  "paymentDetails": {
    "card": "Amex",
    "ccNumber": 12345,
    "expiry": "2001-04-01"
  }
}
```

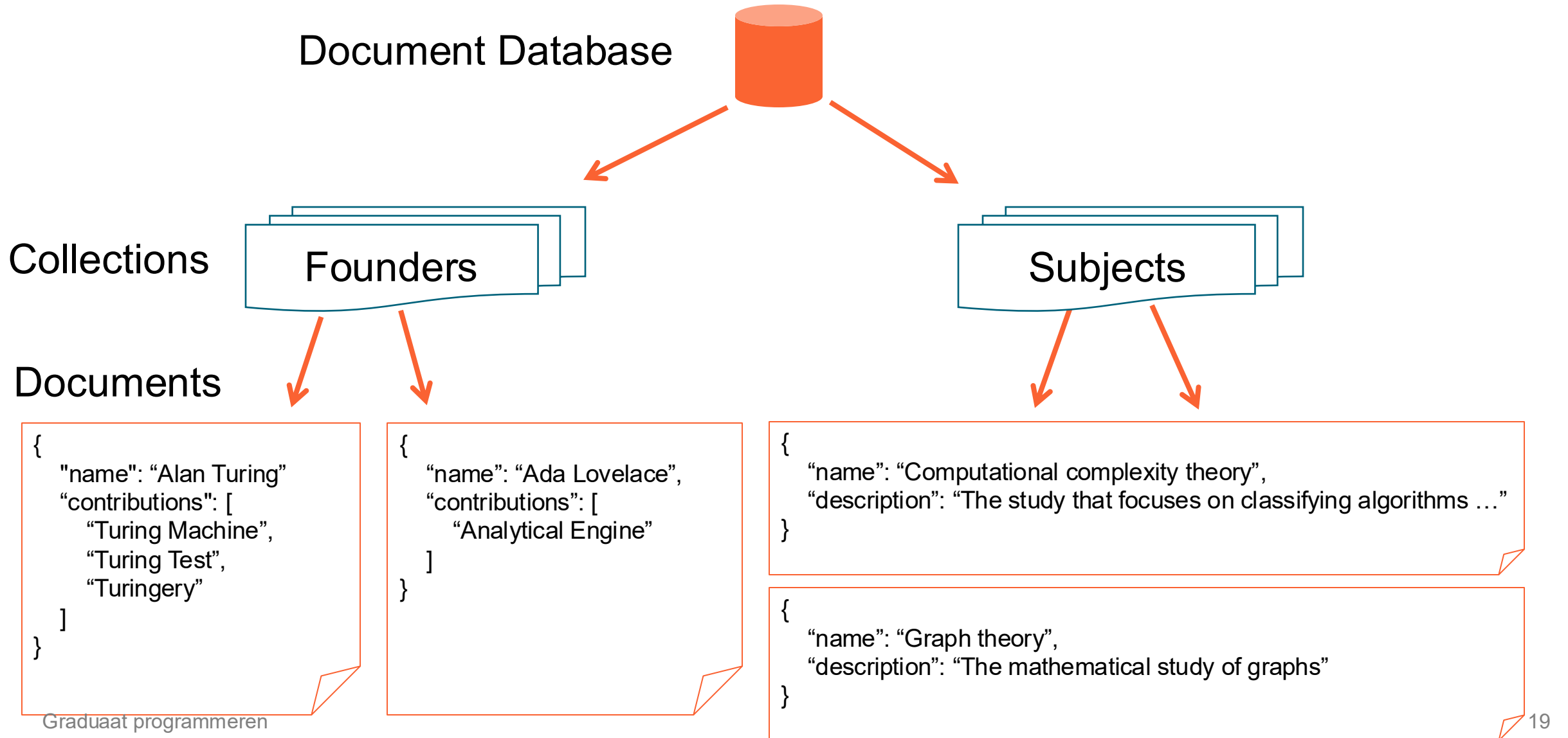

Relationeel vs. document

- Webshop met verschillende Apple en Microsoft producten
 - Relationeel → Karakteristieke entiteit
 - Document DB → String veld in het product
- Wat als je wil filteren of gemakkelijk een merk selecteren tijdens het toevoegen van een product
 - Extra collectie Merken aanmaken
 - Geen referentie maar de merken steeds kopiëren in de producten
 - Data samenvoegen is niet efficiënt
 - Duplicatie wordt in veel gevallen verkozen boven referenties

Relationeel vs. document

Relationeel	Document
Schema ligt vast in de DB	Schema wordt optioneel afgedwongen in de DB
Schema is genormaliseerd en vertrekt vanuit de data	Schema opgebouwd vanuit usecases en kan duplicaten bevatten
Aanpassingen in het schema zijn doorgaan kostelijke operaties (hefboomeffect)	Aanpassingen in het schema zijn eenvoudig en snel
Voldoet aan de ACID voorwaarden	Voldoet niet noodzakelijk aan ACID voorwaarden
Kan doorgaans niet gedistribueerd geïmplementeerd worden	Horizontaal schaalbaar en gedistribueerd
Joins zijn cruciaal	Joins bestaan niet , samen horende data wordt in één keer bewaard

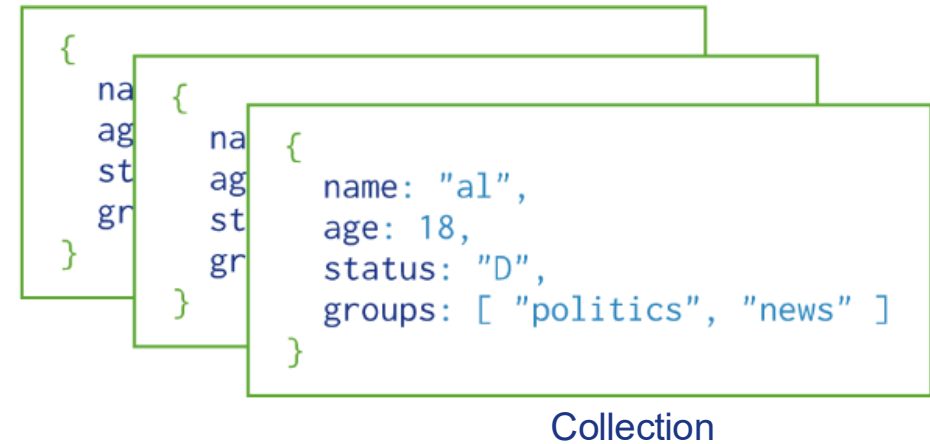
Collections & documents



Collections & documents

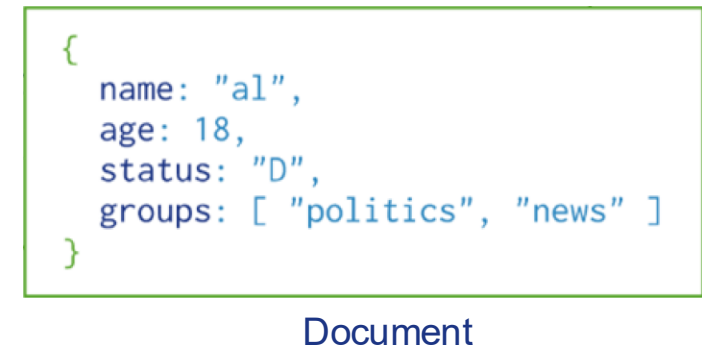
Collections:

- Analooq aan tabellen
 - Bevatten verschillende documenten
- Impliciet aangemaakt tijdens een insert
- Geen DDL statements nodig
- Geen schema
 - Optionele validatie is (soms) mogelijk
- De naam staat in het meervoud



Documents:

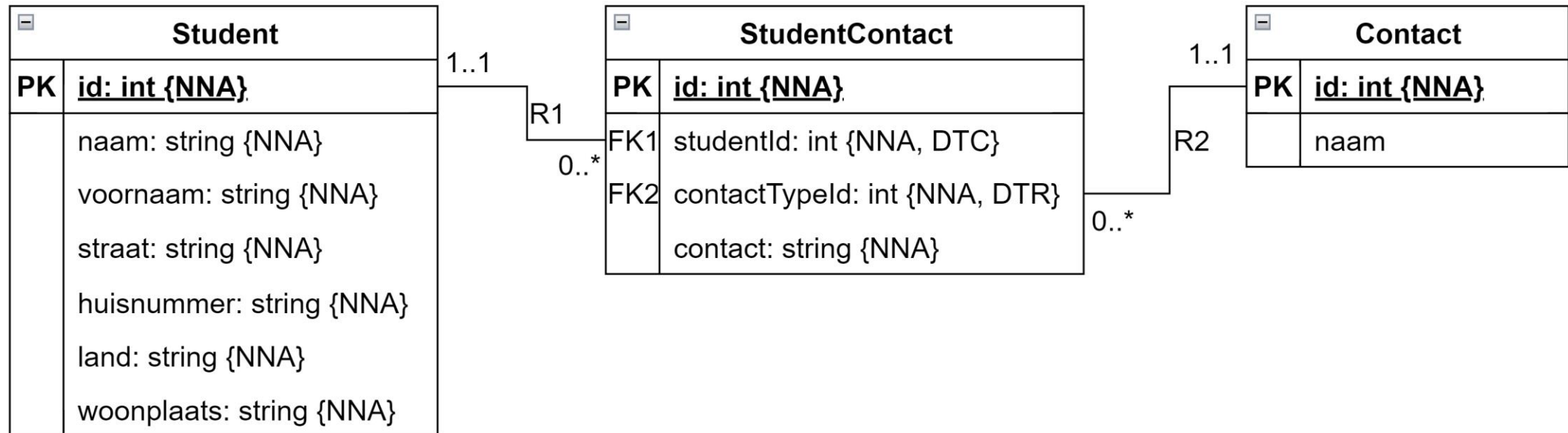
- Analooq aan een rij



Document databases

Modelleren & schema definitie
Embedding vs referencing

Modelleervoorbeeld: Persoonsgegevens



Modelleervoorbeeld: Persoonsgegevens

- Het adres heeft een **één-op-één** relatie met de student
 - Eén adres is uniek per student

Modelleervoorbeeld: Persoonsgegevens

- Het adres heeft een **één-op-één** relatie met de student
 - Eén adres is uniek per student
- embed in een *object*

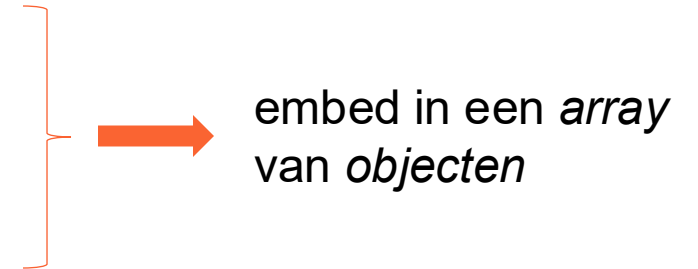
```
{  
  "naam": "Holmes",  
  "voornaam": "Sherlock",  
  "adres": {  
    "straat": "Baker Street",  
    "huisnummer": "221B",  
    "woonplaats": "London",  
    "land": "United Kingdom"  
  }  
}
```


Modelleervoorbeeld: Persoonsgegevens

- Het ERD bevat een veel-op-veel relatie voor de contactgegevens
 - Realiteit: **één-op-enkele** (one-to-few)
 - Contactgegevens worden steeds opgehaald als onderdeel van de persoonsgegevens

Modelleervoorbeeld: Persoonsgegevens

- Het ERD bevat een veel-op-veel relatie voor de contactgegevens
 - Realiteit: **één-op-enkele** (one-to-few)
 - Contactgegevens worden steeds opgehaald als onderdeel van de persoonsgegevens



Modelleervoorbeeld: Persoonsgegevens

- Het ERD bevat een veel-op-veel relatie voor de contactgegevens
 - Realiteit: **één-op-enkele** (one-to-few)
 - Contactgegevens worden steeds opgehaald als onderdeel van de persoonsgegevens

→ embed in een *array*
van *objecten*

```
{
  "naam": "Holmes",
  "voornaam": "Sherlock",
  "adres": {
    "straat": "Baker Street",
    "huisnummer": "221B",
    "woonplaats": "London",
    "land": "United Kingdom"
  },
  "contactGegevens": [
    {
      "type": "email",
      "contact": "serlock@holmes.com"
    },
    {
      "type": "phone",
      "contact": "328.734.8325"
    },
    ...
  ]
}
```

Modelleervoorbeeld: Persoonsgegevens

- Het ERD bevat een veel-op-veel relatie voor de contactgegevens
 - Realiteit: **één-op-enkele** (one-to-few)
 - Contactgegevens worden steeds opgehaald als onderdeel van de persoonsgegevens
- embed in een *array* van *objecten*

Een één-op-enkele relatie is een relatie waarbij de kardinaliteit aan veel kant laag is. Een student heeft slechts enkele emailadressen en dit is dus een één-op-enkele relatie.

```
{  
  "naam": "Holmes",  
  "voornaam": "Sherlock",  
  "adres": {  
    "straat": "Baker Street",  
    "huisnummer": "221B",  
    "woonplaats": "London",  
    "land": "United Kingdom"  
  },  
  "contactGegevens": [  
    {  
      "type": "email",  
      "contact": "serlock@holmes.com"  
    },  
    {  
      "type": "phone",  
      "contact": "328.734.8325"  
    },  
    ...  
  ]  
}
```

Schema definiëren

Document databases hebben niet altijd een schema

- Een goede structuur is echter wel belangrijk
- Twee opties
 - Schema wordt gegarandeerd door de programmeur
 - Schema wordt (indien mogelijk) vastgelegd op database niveau

Schema definiëren

Document databases hebben niet altijd een schema

- Een goede structuur is echter wel belangrijk
- Twee opties
 - Schema wordt gegarandeerd door de programmeur
 - **Schema wordt (indien mogelijk) vastgelegd op database niveau aan de hand van validatieregels**

We gebruiken de JSON schema validatie die ingebouwd is in MongoDB. Aan het einde van de presentatie kan je informatie vinden over hoe je een collectie kan aanmaken die gevalideerd wordt door een opgegeven schema.

Modelleervoorbeeld: Persoonsgegevens

- De validatieregels beschrijven de data die toegestaan is in de database en de structuur hiervan  het schema
- De validatieregels worden beschreven als een JSON Schema

Modelleervoorbeeld: Persoonsgegevens

- De *description* property is **optioneel** en geeft een beschrijving van de property/het object

```
{  
  "description": "Dit document beschrijft het ...",  
  "bsonType": "object",  
  "required": [  
    "naam",  
    "voornaam"  
  ],  
  "properties": {  
    "naam": {  
      "bsonType": "string",  
      "description": "De achternaam van de ..."  
    },  
    "voornaam": {  
      "bsonType": "string",  
      "description": "De voornaam van de ..."  
    }  
  }  
}
```


Modelleervoorbeeld: Persoonsgegevens

- De *description* property is **optioneel** en geeft een beschrijving van de property/het object
- Via *required* kan aangegeven worden welke properties verplicht zijn

```
{  
  "description": "Dit document beschrijft het ...",  
  "bsonType": "object",  
  "required": [  
    "naam",  
    "voornaam"  
  ],  
  "properties": {  
    "naam": {  
      "bsonType": "string",  
      "description": "De achternaam van de ..."  
    },  
    "voornaam": {  
      "bsonType": "string",  
      "description": "De voornaam van de ..."  
    }  
  }  
}
```

Modelleervoorbeeld: Persoonsgegevens

- De *description* property is **optioneel** en geeft een beschrijving van de property/het object **en de validatieregels**
- Via *required* kan aangegeven worden welke properties verplicht zijn
- Via *bsonType* kan het type van het een property aangegeven worden
 - *object* | *array* | *string* | *number* | *date* | *int* | *decimal* | ...
 - Volledige lijst op <https://www.mongodb.com/docs/manual/reference/bson-types/>

```
{  
  "description": "Dit document beschrijft het ...",  
  "bsonType": "object",  
  "required": [  
    "naam",  
    "voornaam"  
  ],  
  "properties": {  
    "naam": {  
      "bsonType": "string",  
      "description": "De achternaam van de ..."  
    },  
    "voornaam": {  
      "bsonType": "string",  
      "description": "De voornaam van de ..."  
    }  
  }  
}
```

Modelleervoorbeeld: Persoonsgegevens

- Properties kunnen geneste objecten bevatten
 - Array
 - Object

```
{
  ...
  "properties": {
    ...
    "contactGegevens": {
      "bsonType": "array",
      "items": {
        "bsonType": "object",
        "required": [
          "type",
          "contact"
        ],
        "properties": {
          "type": {
            "bsonType": "string"
          },
          "contact": {
            "bsonType": "string"
          }
        }
      }
    }
  }
}
```

Modelleervoorbeeld: Persoonsgegevens

- Properties kunnen geneste objecten bevatten
 - Array
 - Object
- De structuur is volledig gelijk (voor objecten)

```
{
  ...
  "properties": {
    ...
    "contactGegevens": {
      "bsonType": "array",
      "items": {
        "bsonType": "object",
        "required": [
          "type",
          "contact"
        ],
        "properties": {
          "type": {
            "bsonType": "string"
          },
          "contact": {
            "bsonType": "string"
          }
        }
      }
    }
  }
}
```

Modelleervoorbeeld: Persoonsgegevens

- Properties kunnen geneste objecten bevatten
 - Array
 - Object
- De structuur is volledig gelijk (voor objecten)
- Voor een array gebruik je *items* waaraan je een *object* koppelt

```
{
  ...
  "properties": {
    ...
    "contactGegevens": {
      "bsonType": "array",
      "items": {
        "bsonType": "object",
        "required": [
          "type",
          "contact"
        ],
        "properties": {
          "type": {
            "bsonType": "string"
          },
          "contact": {
            "bsonType": "string"
          }
        }
      }
    }
  }
}
```

Modelleervoorbeeld: Persoonsgegevens

- Properties kunnen geneste objecten bevatten
 - Array
 - Object
- De structuur is van belang (voor de objecten)
- Voor een array gebruik je *items* waaraan je een *object* koppelt

Het schema dat we hier gebouwd hebben moet in MongoDB ingegeven worden als {"\$jsonSchema": SCHEMA} (zie laatste dia van de presentatie).

```
{
  ...
  "properties": {
    ...
    "contactGegevens": {
      "bsonType": "array",
      "items": {
        "bsonType": "object",
        "properties": {
          "contact": {
            "bsonType": "string"
          }
        }
      }
    }
  }
}
```

```

{
  "description": "Dit document beschrijft het ...",
  "bsonType": "object",
  "required": ["naam", "voornaam", "contactGegevens"],
  "properties": {
    "naam": {
      "bsonType": "string",
      "description": "De achternaam van de ..."
    },
    "voornaam": {
      "bsonType": "string",
      "description": "De voornaam van de ..."
    },
    "contactGegevens": {
      "bsonType": "array",
      "items": {
        "bsonType": "object",
        "required": ["type", "contact"],
        "properties": {
          "type": { "bsonType": "string" },
          "contact": { "bsonType": "string" }
        }
      }
    }
  }
}

```

```

{
  "naam": "Donald",
  "voornaam": "Knuth",
  "contactGegevens": [
    {
      "type": "email",
      "contact": "donald.knuth@stanford.edu"
    },
    {
      "type": "website",
      "contact": "www-cs-faculty.stanford.edu/~knuth"
    }
  ]
}

```

Modelleervoorbeeld: Vakken

- Een student kan meerdere vakken volgen, in een vak zitten altijd meerdere studenten
 - Veel-op-veel
- Twee opties
 - Embedden
 - Data wordt bewaard als kind van een ander document
 - Verwijzingen
 - Data wordt bewaard in een aparte collectie
 - Verwijzing via objectId (automatisch aangemaakt bij create)
 - **Verwijzingen != joins / FK's**

Modelleervoorbeeld: Vakken

- Een student kan meerdere vakken volgen, in een vak zitten altijd meerdere studenten {
 - Veel-op-veel
- Twee opties
 - **Embedden**
 - Vakken enkel beschikbaar als onderdeel van een student
 - Vakken moeten bij **elke** student **gedupliceerd** worden
 - Een overzicht van vakken is niet efficiënt te bouwen
 - De properties van een vak kunnen niet eenvoudig aangepast worden
 - Een document in de students collectie bevat alle informatie over die student
 - Verwijzingen

```
...
"properties": {
  ...
  "vakken": {
    "bsonType": "array",
    "items": {
      "bsonType": "object",
      "required": ["name", "ectsCredits"],
      "properties": {
        "name": {"bsonType": "string"},
        "ectsCredits": {"bsonType": "int"}
      }
    }
  }
}
```

Modelleervoorbeeld: Vakken

- Een student kan meerdere vakken volgen, in een vak zitten altijd meerdere studenten
 - Veel-op-veel
- Twee opties
 - **Embedden**
 - Vakken enkel beschikbaar als onderdeel van een student
 - Vakken moeten bij **elke** student **gedupliceerd** worden
 - Een overzicht van vakken is niet efficiënt te bouwen
 - De properties van een vak kunnen niet eenvoudig aangepast worden
 - Een document in de students collectie bevat alle informatie over die student
 - Verwijzingen

```
{
  "vakken": [
    {
      "name": "Algorithms ...",
      "ectsCredits": 5
    },
    {
      "name": "Automata ...",
      "ectsCredits": 4
    }
  ]
}
```

Modelleervoorbeeld: Vakken

- Een student kan meerdere vakken volgen, in een vak zitten altijd meerdere studenten
 - Veel-op-veel
- Twee opties
 - Embedden
 - **Referencing (Verwijzingen)**
 - Vakken worden één keer bewaard
 - Om de overzichtspagina van een student te bouwen (inclusief vakken) zijn meerdere queries nodig (of een view met *\$lookup* in *MongoDB*)
 - Een vak aanpassen is eenvoudig
 - De documenten in de students collectie zijn minder complex

```
{  
  ...  
  "properties": {  
    ...  
    "vakken": {  
      "bsonType": "array",  
      "items": {  
        "bsonType": "objectId"  
      }  
    }  
  }  
}
```

Modelleervoorbeeld: Vakken

- Een student kan meerdere vakken volgen, in een vak zitten altijd meerdere studenten
 - Veel-op-veel
- Twee opties
 - Embedden
 - **Referencing (Verwijzingen)**
 - Vakken worden één keer bewaard
 - Om de overzichtspagina van een student te bouwen (inclusief vakken) zijn meerdere queries nodig (of een view met *\$lookup* in *MongoDB*)
 - Een vak aanpassen is eenvoudig
 - De documenten in de students collectie zijn minder complex

```
{  
  "naam": "Donald",  
  "voornaam": "Knuth",  
  "contactGegevens": [...],  
  "vakken": [  
    {"$oid": "61741c9cbc9ec583c836170a"},  
    {"$oid": "61741c9cbc9ec583c836170b"}  
  ]  
}
```



Modelleervoorbeeld: Vakken

- Een student kan meerdere vakken volgen, in een vak zitten altijd meerdere studenten

- Veel-op-veel

- Twee opties

- Embedden

- **Referencing (Verwijzingen)**

- Vakken worden één keer bewaard
 - Om de overzichtspagina van een student bouwen (inclusief vakken) zijn meerder nodig (of een view met *\$lookup* in MongoDB)
 - Een vak aanpassen is eenvoudig
 - De documenten in de students collectie minder complex

```
{  
  "naam": "Donald",  
  "voornaam": "Knuth",  
  "contactGegevens": [...],  
  "vakken": [  
    {"$oid": "61741c9cbc9ec583c836170a"},  
    {"$oid": "61741c9cbc9ec583c836170b"}  
  ]  
}
```

Referenties naar andere documenten moeten
ingegeven worden als een document met één property
\$oid.

Modeleervoorbeeld: Papers

- Een student schrijft doorheen een opleiding meerdere papers
 - Eén-op-veel
- **Optie 1: Referentie in paper**
 - Analooq aan SQL
 - Zelden/nooit alleen opgevraagd, steeds als onderdeel van een student
 - Het aantal papers is ongelimiteerd
 - Meerdere queries nodig
- Optie 2: Embedded in student
- Optie 3: Referentie in student

```
{  
  "description": "Paper",  
  "bsonType": "object",  
  "required": [  
    "title", "abstract",  
    "grade", "pdfURL",  
    "student"  
  ],  
  "properties": {  
    "title": {"bsonType": "string"},  
    "abstract": {"bsonType": "string"},  
    "grade": {"bsonType": "int"},  
    "pdfURL": {"bsonType": "string"},  
    "student": {"bsonType": "objectId"}  
  }  
}
```

Modeleervoorbeeld: Papers

- Een student schrijft doorheen een opleiding meerdere papers
 - Eén-op-veel
- **Optie 1: Referentie in paper**
 - Analooog aan SQL
 - Zelden/nooit alleen opgevraagd, steeds als onderdeel van een student
 - Het aantal papers is ongelimiteerd
 - Meerdere queries nodig
- Optie 2: Embedded in student
- Optie 3: Referentie in student

```
{  
  "title": "Sorting algorithms",  
  "abstract": "In this paper we ...",  
  "grade": 17,  
  "pdfURL": "/blobs/453REER62.pdf",  
  "student": {  
    "$oid": "61741c9cbc9ec583c836170b"  
  }  
}
```

Modeleervoorbeeld: Papers

- Een student schrijft doorheen een opleiding meerdere papers
 - Eén-op-veel
- Optie 1: Referentie in paper
- **Optie 2: Embedded in student**
 - Papers worden bewaard samen met de studenten
 - Samen nodig → samen bewaren
 - Aantal papers is beperkt
 - Max grootte van een document is **16MB**
 - Veel papers → Te groot student document
- Optie 3: Referentie in student

```
{
  "properties": {
    "papers": {
      "bsonType": "array",
      "items": {
        "bsonType": "object",
        "required": [
          "title", "abstract",
          "grade", "pdfURL"
        ],
        "properties": {
          "title": { "bsonType": "string" },
          "abstract": { "bsonType": "string" },
          "grade": { "bsonType": "int" },
          "pdfURL": { "bsonType": "string" }
        }
      }
    }
  }
}
```


Modeleervoorbeeld: Papers

- Een student schrijft doorheen een opleiding meerdere papers
 - Eén-op-veel
- Optie 1: Referentie in paper
- **Optie 2: Embedded in student**
 - Papers worden bewaard samen met de studenten
 - Samen nodig → samen bewaren
 - Aantal papers is beperkt
 - Max grootte van een document is **16MB**
 - Veel papers → Te groot student document
- Optie 3: Referentie in student

```
{  
  "naam": "Donald",  
  "voornaam": "Knuth",  
  "contactGegevens": [...],  
  "vakken": [...],  
  "papers": [  
    {  
      "title": "Sorting algorithms",  
      "abstract": "In this paper we ...",  
      "grade": 17,  
      "pdfURL": "/blobs/453REER62.pdf"  
    }  
  ]  
}
```

Modeleervoorbeeld: Papers

- Een student schrijft doorheen een opleiding meerdere papers

- Eén-op-veel

- Optie 1: Referentie

- **Optie 2: Embed**

- Papers worden bewaard door studenten

- Samen nodig → samen bewaren

- Aantal papers is beperkt

- Max grootte van een document is **16MB**
 - Veel papers → Te groot student document

- Optie 3: Referentie in student

Veel-op-duizenden relaties moeten referencing gebruiken aan de duizenden kant van de relatie anders wordt het document te groot. Voorbeelden van dit soort relaties zijn commentaren op social media posts of de inwoners van een stad.

```
{
  "author": "John Doe",
  "title": "Sorting algorithms",
  "abstract": "In this paper we ...",
  "grade": 17,
  "pdfURL": "/blobs/453REER62.pdf"
}
```

Modeleervoorbeeld: Papers

- Een student schrijft doorheen een opleiding meerdere papers
 - Eén-op-veel
- Optie 1: Referentie in paper
- Optie 2: Embedded in student
- **Optie 3: Referentie in student**

De locatie van de referentie is afhankelijk van de usecase en hoe de programmeur dingen organiseert.

Via arrays kunnen referenties aan de één-kant van een één-op-veel relatie geplaatst worden, natuurlijk kunnen we ook een referentie aan de veel kan van een relatie plaatsen (zoals in SQL).

```
{  
  "naam": "Donald",  
  "voornaam": "Knuth",  
  "contactGegevens": [...],  
  "vakken": [...],  
  "papers": [  
    {  
      "title": "Sorting algorithms",  
      "abstract": "In this paper we ...",  
      "grade": 17,  
      "pdfURL": "/blobs/453REER62.pdf"  
    }  
  ]  
}
```

Modeleervoorbeeld: Papers

- Een student schrijft doorheen een opleiding meerdere papers
 - Eén-op-veel
- Optie 1: Referentie in paper
- Optie 2: Embedded in student
- **Optie 3: Referentie in student**
 - Aantal papers is onbeperkt
 - Papers zijn een onderdeel van een student document

```
{
  "description": "Dit document beschrijft ...",
  "bsonType": "object",
  "required": [
    "naam", "voornaam",
    "contactGegevens", "vakken",
    "papers"
  ],
  "properties": {
    "papers": {
      "bsonType": "array",
      "items": { "bsonType": "objectId" }
    }
  }
}
```

Modeleervoorbeeld: Papers

- Een student schrijft doorheen een opleiding meerdere papers
 - Eén-op-veel
- Optie 1: Referentie in paper
- Optie 2: Embedded in student
- **Optie 3: Referentie in student**
 - Aantal papers is onbeperkt
 - Papers zijn een onderdeel van een student document

```
{  
  "naam": "Donald",  
  "voornaam": "Knuth",  
  "contactGegevens": [...],  
  "vakken": [...],  
  "papers": [  
    {"$oid": "61741c9cbc9ec583c836170a"},  
    {"$oid": "61741c9cbc9ec583c836170b"}  
  ]  
}
```

Embedded vs. referenced

Embed	Reference
Eén query	Verschillende queries, of een extra view dat gebouwd is met de trage <i>\$lookup</i> operator
Ideaal voor één-op-één relaties	Meestal nodig voor veel-op-veel & één-op-veel relaties
Aantal embedded documenten is gelimiteerd (maximaal 16MB per document)	Aantal gerefereerde documenten is praktisch ongelimiteerd, er zijn ongeveer 0,5 miljoen referenties nodig om aan 16MB te geraken
Data die samen opgevraagd wordt en samen geüpdatet wordt samen bewaard	Data die (regelmatig) individueel aangepast moet worden, wordt in aparte collecties bewaard

Document databases

Special topics

Overerving

- NULL waarden moeten niet ingegeven worden en dus geen verloren ruimte → Single table aanpak
- Via de *anyOf* property kunnen we meerdere objecten toestaan in één collectie

```
{
  "anyOf": [
    {
      "description": "Buitenlandse student",
      "bsonType": "object",
      "required": [...],
      "properties": {...}
    },
    {
      "description": "Student",
      "bsonType": "object",
      "required": [...],
      "properties": {...}
    }
  ]
}
```


Recursie

- Optie 1: Hiërarchie overnemen via embedding
 - Goed in de meeste gevallen
 - Alle data in één document/query
 - Te grote hiërarchie → optie 2
- Optie 2: Referentie gebruiken
 - Een attribuut met een referentie naar het bovenliggende document
 - Een array met onderliggende documenten

 Validatie onmogelijk

 Validatie mogelijk

Document databases

Aan de slag met MongoDB

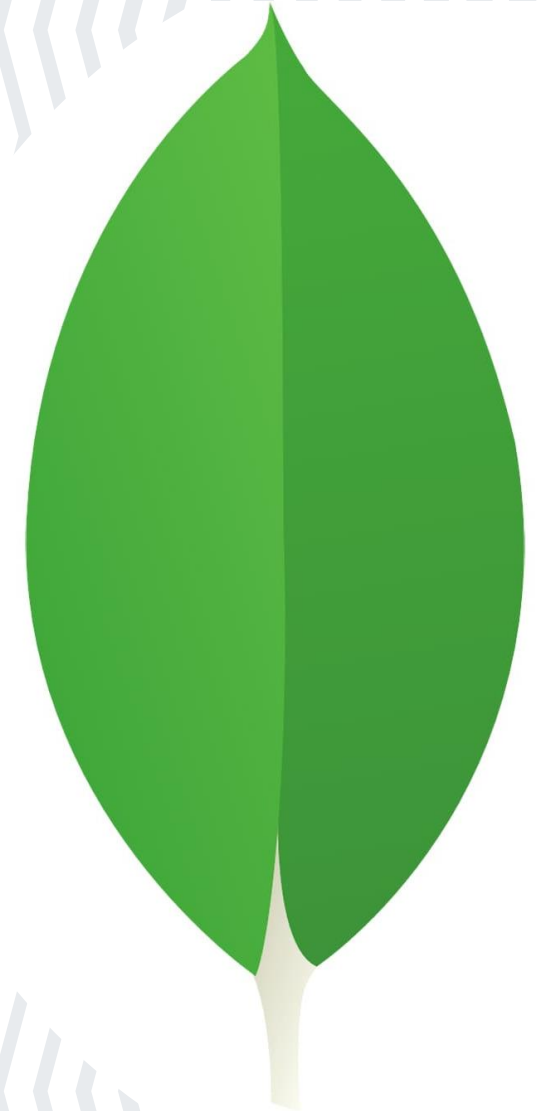
MongoDB

- Eén van de meest populaire document databases
- **Optionele ondersteuning voor schema validatie via een JSON schema**
- Gebruikt BSON, een binair formaat dat enkele uitbreidingen toevoegt aan JSON



MongoDB Community Server

- Open-source database management system (DBMS) voor document-georiënteerde data
- Beheert databases, die collecties van documenten bevatten
- Downloaden van <https://www.mongodb.com/try/download/community>



MongoDB Community Server

- Open-source database management system (DBMS) voor document-georiënteerde data

- Beheert data documenten

Kies tijdens de installatie voor de “Complete” optie. Gebruik vervolgens de default instellingen in het “Service Configuration” scherm.

- Downloaden van <https://www.mongodb.com/try/download/community>



MongoDB Compass

- GUI om data op te vragen en in te voegen
- Verbind met een instantie van de MongoDB (community) server
- Ideaal voor gebruiker met minder technische kennis
- Kan een schema genereren op basis van ingevoegde data
- Downloaden vanop <https://www.mongodb.com/try/download/compass> (maar is ook inbegrepen in de community server)



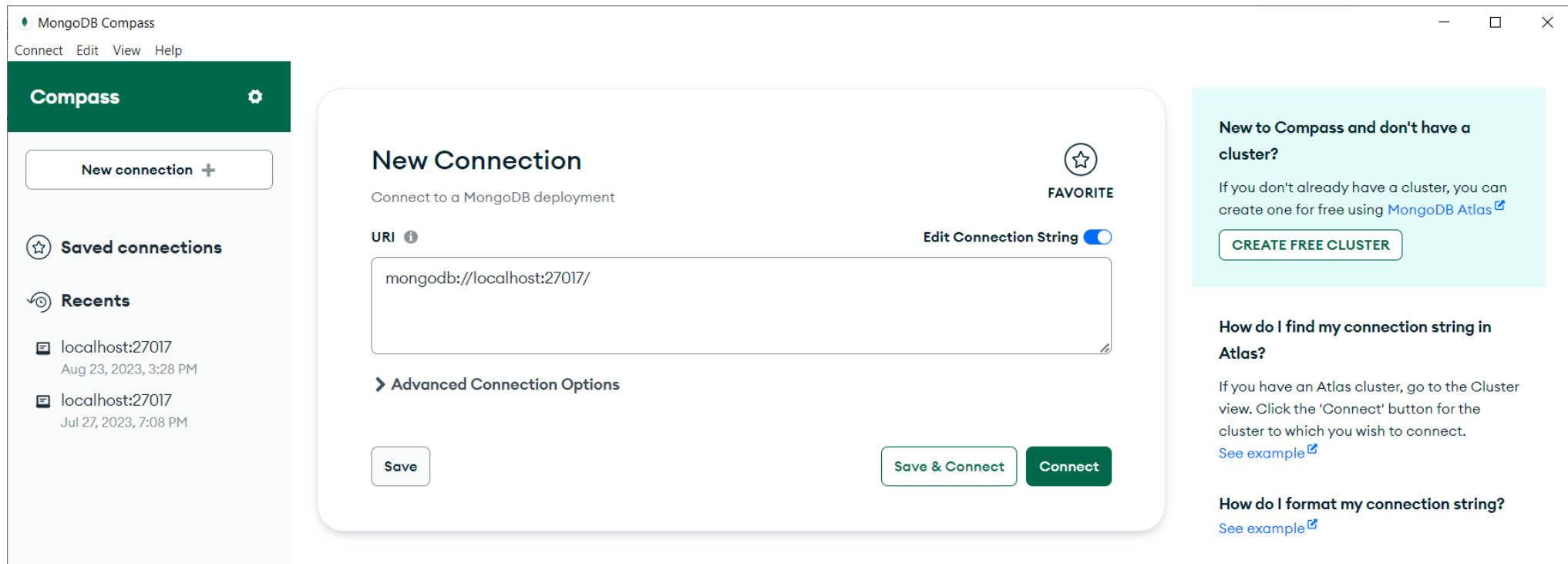
MongoDB Shell

- CLI (command line interface) om data in MongoDB op te vragen en aan te passen
- Verbind met een instantie van de MongoDB (community) server
- De beste optie voor technische gebruikers
- **Kan gebruikt worden om een collectie aan te maken met ingebouwde validatie**
- Is ingebouwd in Compass



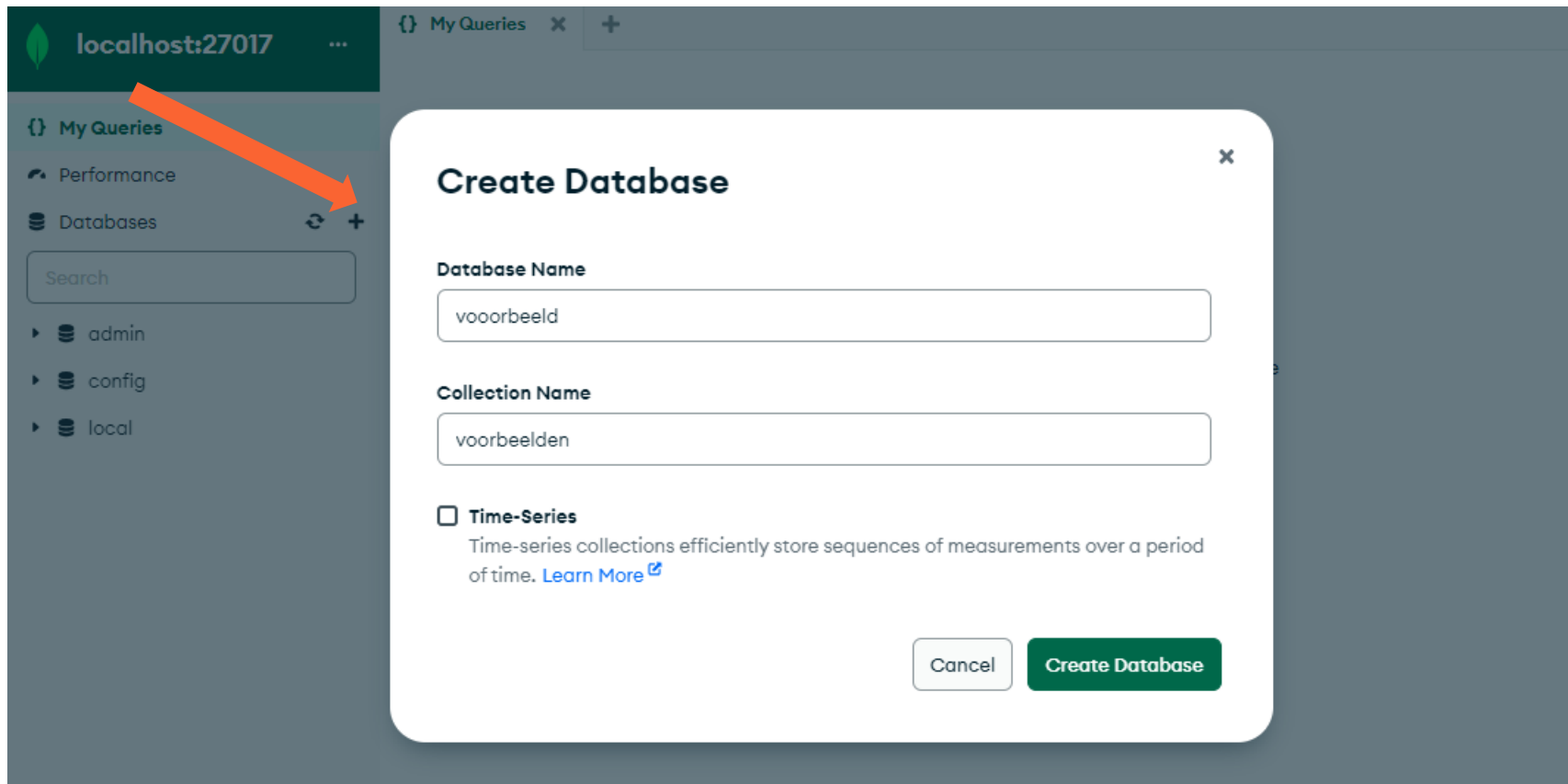
Verbinden met de database

- Open MongoDB Compass
- Vul in het 'New Connection' tabblad de connectiestring 'mongodb://localhost:27017/' in om verbinding te maken met de community server
- Klik op 'Save & Connect'



Database & collectie aanmaken

- Maak één database aan per oefening
- Per database moet je minstens één collectie aanmaken



Validatieregels toevoegen

- Selecteer de collectie in de nieuwe database
- Selecteer het validatie tabblad
- Voeg de code uit modelleervoorbeeld 1 in

